PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Construcción de una API REST con Node.js, Express y MySQL

Presentado a: **Instructora: Yoli Alexandra Guevara Muñoz**

Por Aprendiz: **Sebastián Alejandro Bolaños**

Ficha: **3312932**

Competencia: **BACKEND CON NODE JS**

Tecnólogo en Análisis y Desarrollo de Software
Servicio Nacional de Aprendizaje SENA
Centro de Teleinformática y Producción Industrial
Regional Cauca

Popayán, día **08** de **11** del año **2026**

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Tabla de contenido

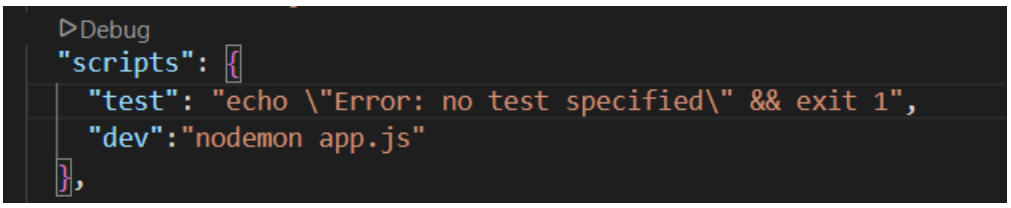
1. Requerimientos técnicos 3
2. Situación problema y base de datos 4
3. Estructura del proyecto 5
    1. Actividades de desarrollo 5
    Actividad 2. Crear funciones de prueba 7
    Actividad 3. Desarrollo del módulo de usuarios 12
    Actividad 4. Desarrollo del módulo de productos 16
    Actividad 5. Integración de rutas y conexión 21
    Actividad 6. Identificación y solución de errores 23
    Actividad 7. Pruebas integrales con Postman 28
    8. Preguntas de análisis y sustentación 33

1. Requerimientos técnicos

- Node.js y npm.
- Express para la creación del servidor y las rutas.
- MySQL2 para la conexión con la base de datos.
- CORS para permitir solicitudes desde otros orígenes.
- Nodemon para reiniciar el servidor durante el desarrollo.
- Postman para probar los endpoints.
- CommonJS: require() y module.exports.
- Clases de JavaScript para implementar los Controllers.

npm init -y
npm install express mysql2 cors
npm install nodemon --save-dev

Configure en package.json el script de desarrollo:



{
"scripts": {

```
  "dev": "nodemon app.js"
 }
```

2. Situación problema y base de datos

La empresa requiere administrar dos entidades: usuarios y productos. Cree una base de datos llamada taller_api y las siguientes tablas:

Tabla usuarios

Campo	Tipo	Restricción
id	INT	PRIMARY KEY, AUTO_INCREMENT
nombre	VARCHAR(100)	NOT NULL
apellido	VARCHAR(100)	NOT NULL
direccion	VARCHAR(200)	Puede ser NULL
telefono	VARCHAR(20)	Puede ser NULL

Base de datos mysql

```sql
drop database if exists taller_api;
create database taller_api;
use taller_api;

create table tabla_usuarios(
id int auto_increment primary key,
nombre varchar(100) not null,
apellido varchar(100) not null,
direccion varchar(200) not null,
telefono varchar(20)
);
```

Tabla productos

Campo	Tipo	Restricción
id	INT	PRIMARY KEY, AUTO_INCREMENT
nombre	VARCHAR(100)	NOT NULL
descripcion	TEXT	Puede ser NULL

precio	DECIMAL(10,2)	NOT NULL
stock	INT	NOT NULL

```sql
drop database if exists taller_api;
create database taller_api;
use taller_api;
```

```sql
create table tabla_producto(
id int auto_increment primary key,
nombre varchar(100) not null,
descripcion text,
precio decimal(10,2) not null,
stock int not null
);
```

3. Estructura del proyecto

Organice los archivos de la siguiente manera. La conexión a MySQL debe permanecer en app.js, tal como se solicita en este taller.

1. Actividades de desarrollo

Actividad 1. Configuración inicial del servidor

Enunciado: Cree un proyecto Node.js e instale las dependencias necesarias. Configure Express para recibir solicitudes HTTP, procesar información JSON y permitir solicitudes desde otros orígenes mediante CORS. El servidor debe ejecutarse por el puerto 3000.

Debe realizar:

1. Inicializar el proyecto con npm init -y.

```
C:\Users\PERSONAL\Desktop\Taller_Api_Res_Backend>npm init -y
Wrote to C:\Users\PERSONAL\Desktop\Taller_Api_Res_Backend\package.json:

{
  "name": "taller_api_res_backend",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

2. Instalar Express, MySQL2, CORS y Nodemon.

```
C:\Users\PERSONAL\Desktop\Taller_Api_Res_Backend>npm install express

up to date, audited 104 packages in 3s

38 packages are looking for funding
  run `npm fund` for details

2 moderate severity vulnerabilities

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
```

```
C:\Users\PERSONAL\Desktop\Taller_Api_Res_Backend>npm install express mysql2 cors

added 77 packages, and audited 78 packages in 7s

33 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\PERSONAL\Desktop\Taller_Api_Res_Backend>
```

```
C:\Users\PERSONAL\Desktop\Taller_Api_Res_Backend>npm install cors

up to date, audited 104 packages in 4s

38 packages are looking for funding
  run `npm fund` for details

2 moderate severity vulnerabilities

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
```

```
C:\Users\PERSONAL\Desktop\Taller_Api_Res_Backend>npm install nodemon --save-dev

added 26 packages, and audited 104 packages in 8s

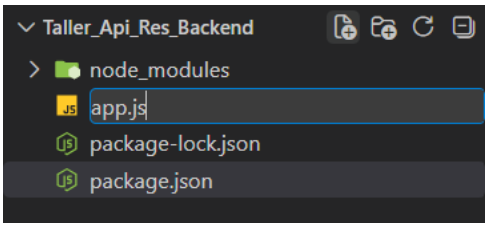
38 packages are looking for funding
  run `npm fund` for details

2 moderate severity vulnerabilities

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
```

3. Crear app.js.



4. Configurar express.json() y cors().

```
const express=require("express")
const cors=require("cors")
const app=express();
const PORT=3000;

app.use(express.json());
app.use(cors());
```

5. Iniciar el servidor y comprobar que se encuentre activo.

```
C:\Users\PERSONAL\Desktop\Taller_Api_Res_Backend_k>npm run dev

> taller_api_res_backend@1.0.0 dev
> nodemon app.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node app.js`
servisor en funcinamiento3000
```

Pruebas y análisis:

- **¿Qué sucede si no instala Express?**

El proyecto no podrá utilizar Express y aparecerá un error indicando que no encuentra el módulo (Cannot find module 'express'). Para solucionarlo, debe instalarlo con npm install express

- **¿Qué sucede si olvida ejecutar express.json() y envía un cuerpo JSON?**

El servidor no podrá interpretar correctamente los datos enviados en formato JSON y req.body aparecerá como undefined.

- **¿Qué sucede si el puerto 3000 ya está ocupado?**

El servidor no podrá iniciarse porque otro programa está utilizando ese puerto. Aparecerá un error indicando que la dirección ya está en uso. Se puede solucionar cerrando el programa que lo utiliza o cambiando el puerto.

- **¿Qué sucede si escribe incorrectamente la ruta de importación de un módulo?**

Node.js no encontrará el archivo o módulo y mostrará un error como Cannot find module. Se debe revisar que la ruta y el nombre del archivo estén escritos correctamente.

Actividad 2. Crear funciones de prueba

Enunciado: Antes de implementar la conexión con la base de datos, la empresa necesita verificar que el servidor reciba solicitudes y responda correctamente. Cree la clase PruebaController y su archivo de rutas PruebaRoutes.

Método solicitado	Comportamiento esperado

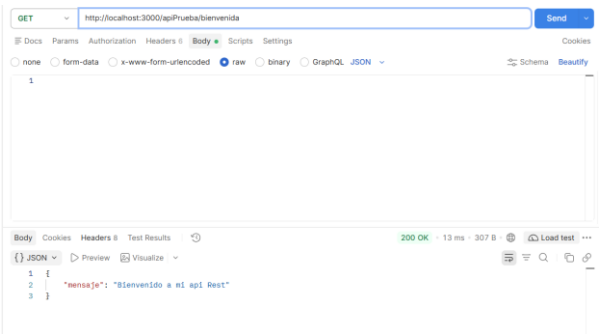
bienvenida	Responder a una solicitud GET con un mensaje de bienvenida.
verificar	Responder con un mensaje y un estado booleano que indique que el servidor funciona.
recibir	Recibir nombre y edad mediante POST y devolver los datos recibidos.
sumar	Recibir dos números mediante POST y devolver el resultado de la suma.

Bienvenida

Responder a una solicitud GET con un mensaje de bienvenida.

```
class PruebaController{
    bienvenida(req,res){
        res.json({"mensaje":"Bienvenido a mi api Rest"})
    }
```

Respuesta

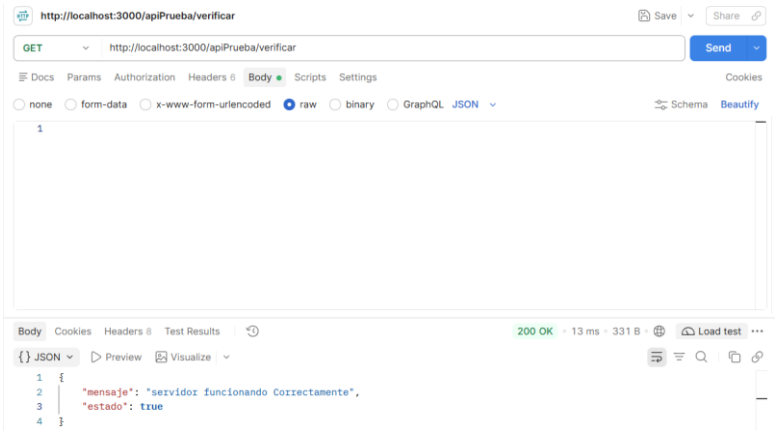


Verificar

Responder con un mensaje y un estado booleano que indique que el servidor funciona.

```
verificar(req,res){
    res.json({"mensaje":"servidor funcionando Correctamente",
        "estado":true
    });
}
```

Respuesta



Recibir

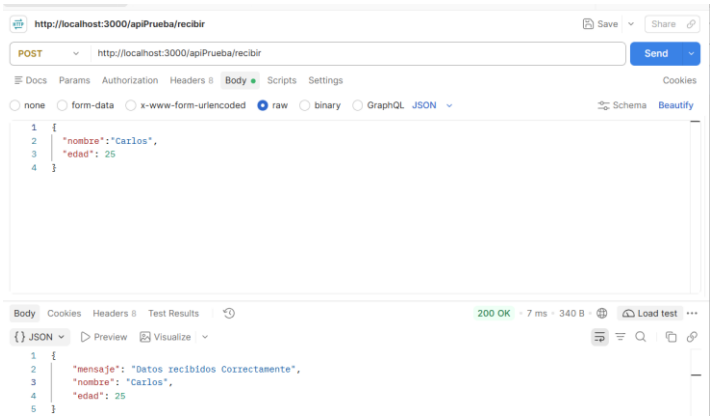
Recibir nombre y edad mediante POST y devolver los datos recibidos.

```
recibir(req,res){
    const nombre= req.body.nombre;
    const edad=req.body.edad

    res.json({"mensaje":"Datos recibidos Correctamente",
        "nombre":nombre,
        "edad":edad
    });
}
```

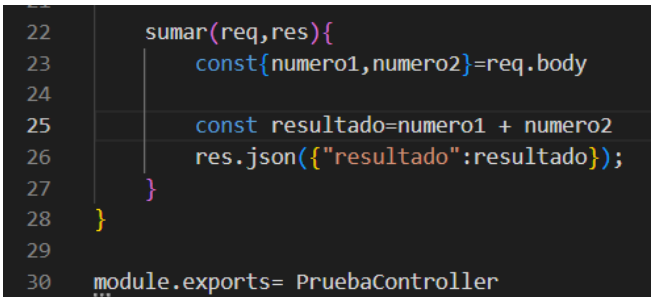
Respuesta

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

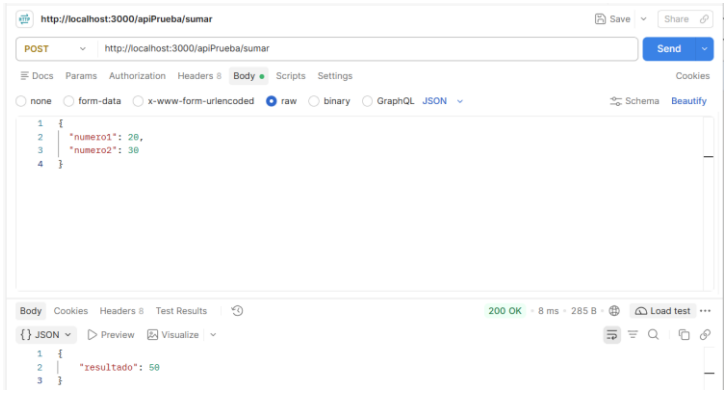


Sumar

Recibir dos números mediante POST y devolver el resultado de la suma.



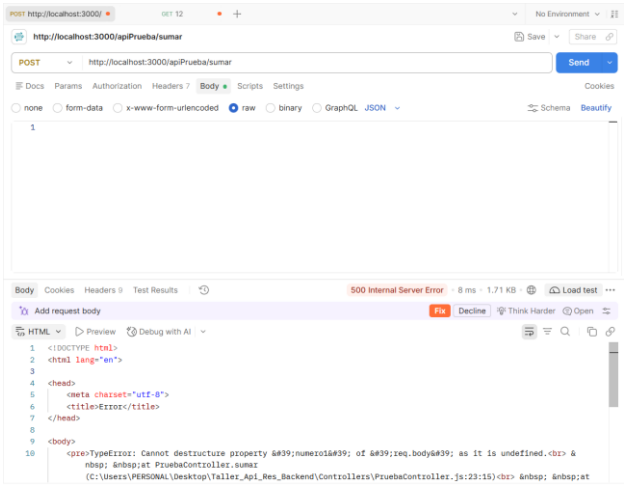
Respuesta



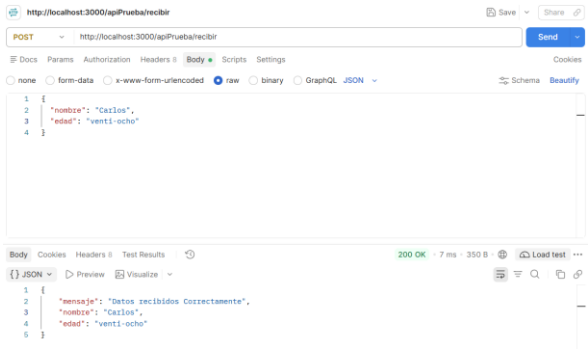
Pruebas y análisis:

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

- **Enviar una solicitud POST sin información.**

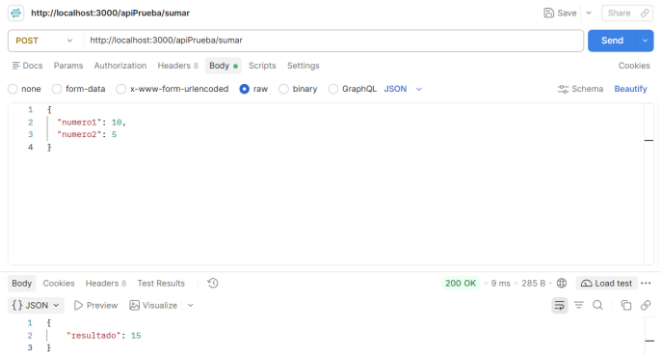


- **Enviar una edad como texto y explicar si el servidor la acepta.**



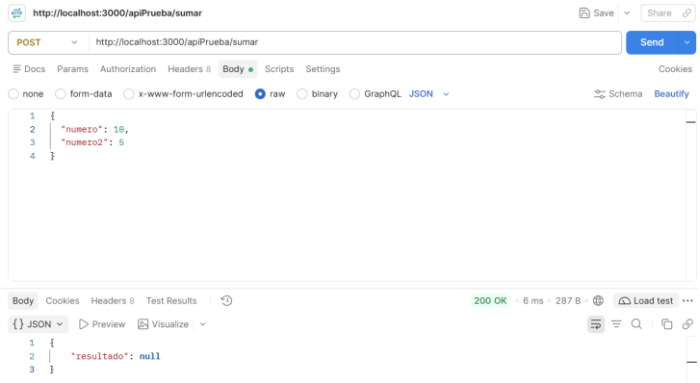
El servidor sí acepta la edad como texto si se envía entre comillas, pero la recibe como una cadena de texto. Si el servidor necesita un número para realizar operaciones matemáticas, debe convertirla a tipo numérico.

- **Enviar números en lugar de cadenas al método sumar.**



Al enviar números en lugar de cadenas de texto al método sumar, el servidor puede realizar la operación matemática correctamente y devolver el resultado de la suma.

- **Omitir uno de los campos requeridos.**



- **Explicar qué sucede al intentar leer req.body cuando req es undefined.**

Si req es undefined, al intentar leer req.body se genera un error porque JavaScript no puede acceder a una propiedad de una variable que no tiene ningún valor. El servidor puede detener la ejecución de esa operación y mostrar un error en la consola.

Actividad 3. Desarrollo del módulo de usuarios

Enunciado: La empresa necesita almacenar la información de las personas que utilizan sus servicios. Utilice la tabla usuarios y desarrolle la clase UsuariosController con los métodos indicados.

Método	Requerimiento

guardar	Registrar un usuario con nombre, apellido, dirección y teléfono.
traer	Consultar todos los usuarios registrados.
buscar	Consultar un usuario mediante su identificador.
actualizar	Modificar la información de un usuario existente.
eliminar	Eliminar un usuario mediante su identificador.

Método HTTP	Endpoint	Operación
POST	/usuarios	Registrar usuario
GET	/usuarios	Consultar todos
GET	/usuarios/:id	Consultar por ID
PUT	/usuarios/:id	Actualizar usuario
DELETE	/usuarios/:id	Eliminar usuario

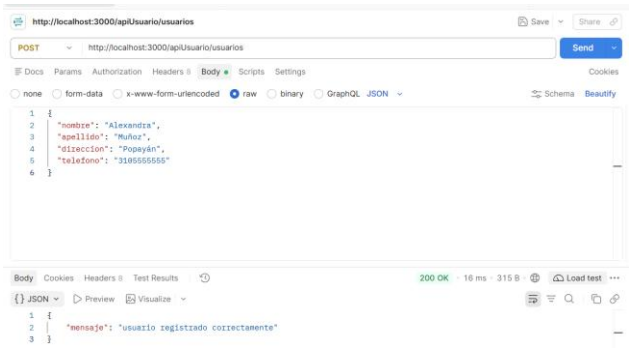
Ejemplo de datos para registrar:

```
{
  "nombre": "Alexandra",
  "apellido": "Muñoz",
  "direccion": "Popayán",
  "telefono": "3105555555"
}
```

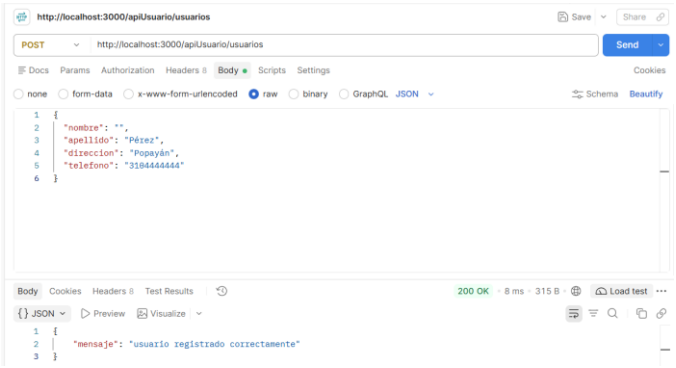
Pruebas y análisis:

- **Registrar un usuario con todos los campos.**

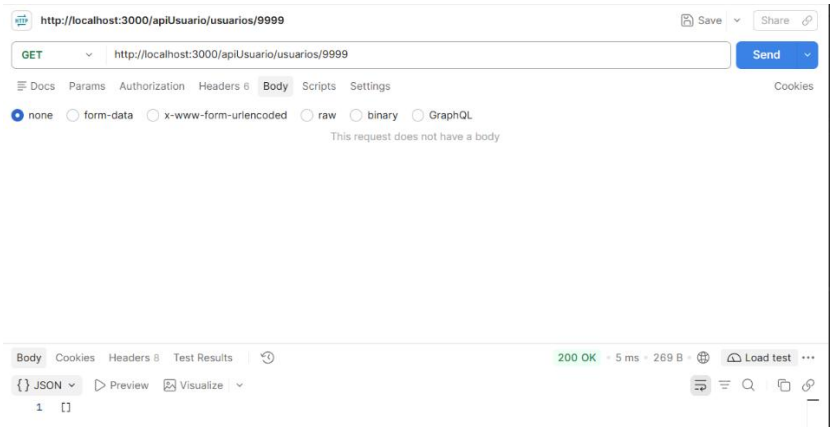
PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS



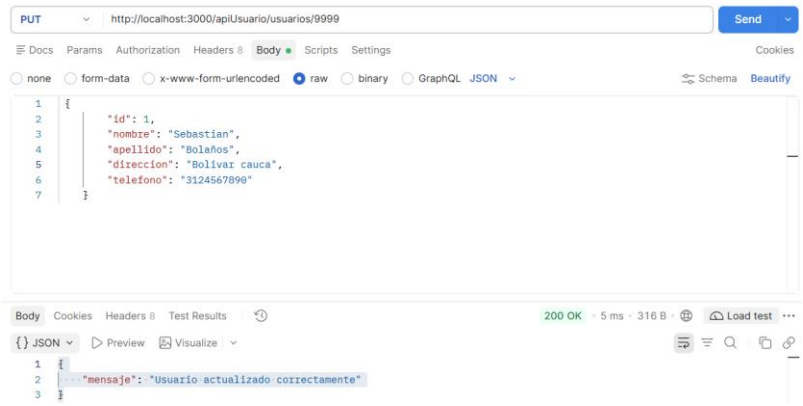
- **Intentar registrar un usuario sin nombre o apellido.**



- **Consultar un identificador que no existe.**

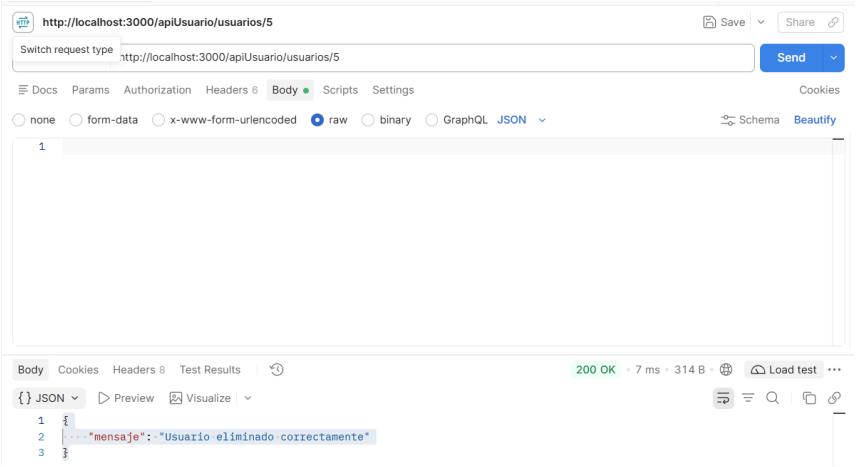


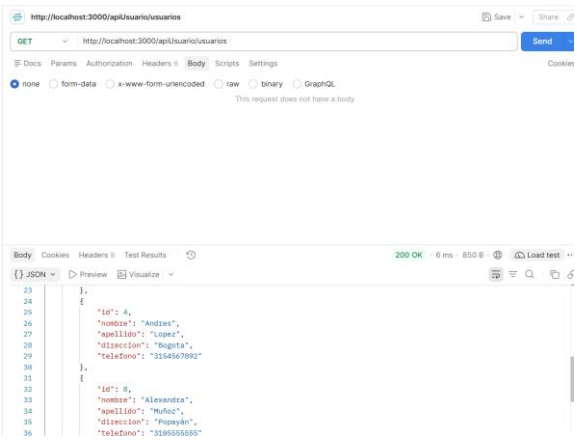
- **Actualizar un identificador inexistente.**



Al intentar actualizar un identificador inexistente, no se modifica ningún usuario porque ese registro no se encuentra en la base de datos. El servidor debería informar que el usuario no existe.

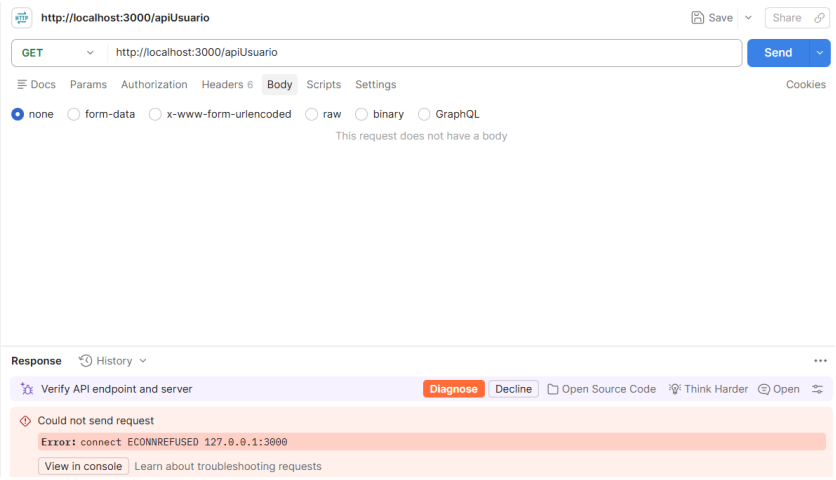
- **Eliminar un usuario y volver a consultar el mismo identificador.**





Después de eliminar el usuario, al consultar nuevamente su identificador no aparecerá ningún registro porque fue eliminado de la base de datos.

- **Desconectar MySQL e intentar consultar usuarios.**



Al desconectar MySQL, el servidor no puede establecer comunicación con la base de datos y no puede consultar los usuarios. Por eso, muestra un error de conexión.

Actividad 4. Desarrollo del módulo de productos

Enunciado: La empresa requiere administrar su inventario de productos tecnológicos. Utilice la tabla productos y desarrolle la clase ProductosController.

Método	Requerimiento
guardar	Registrar nombre, descripción, precio y stock.
traer	Consultar todos los productos.
buscar	Consultar un producto por ID.
actualizar	Modificar los datos de un producto existente.
eliminar	Eliminar un producto por ID.
agotados	Consultar productos cuyo stock sea igual a cero.
bajoStock	Consultar productos cuyo stock sea inferior a cinco unidades.

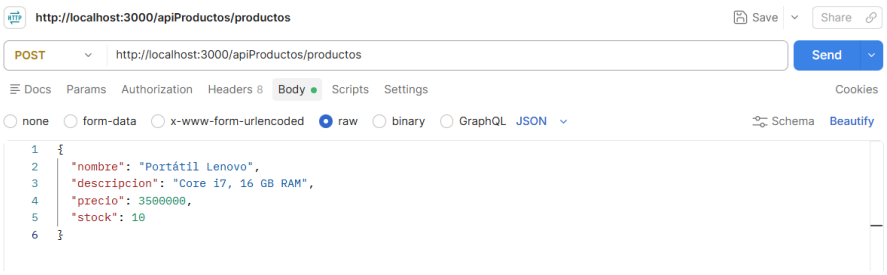
Método HTTP	Endpoint	Operación
POST	/productos	Registrar producto
GET	/productos	Consultar todos
GET	/productos/:id	Consultar por ID
PUT	/productos/:id	Actualizar producto
DELETE	/productos/:id	Eliminar producto
GET	/productos/agotados	Consultar stock igual a cero
GET	/productos/bajo-stock	Consultar stock menor que cinco

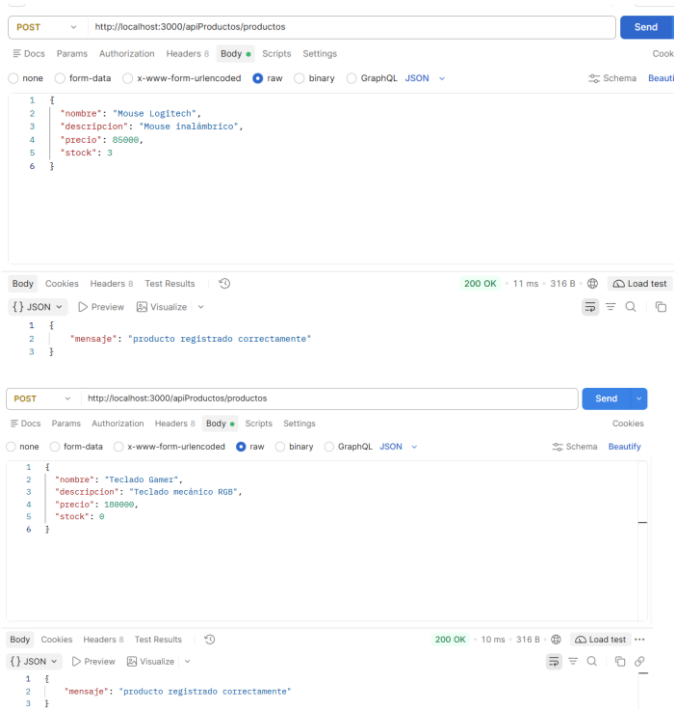
Ejemplo de datos para registrar:

```
{
  "nombre": "Portátil Lenovo",
  "descripcion": "Core i7, 16 GB RAM",
  "precio": 3500000,
  "stock": 10
}
```

Pruebas y análisis:

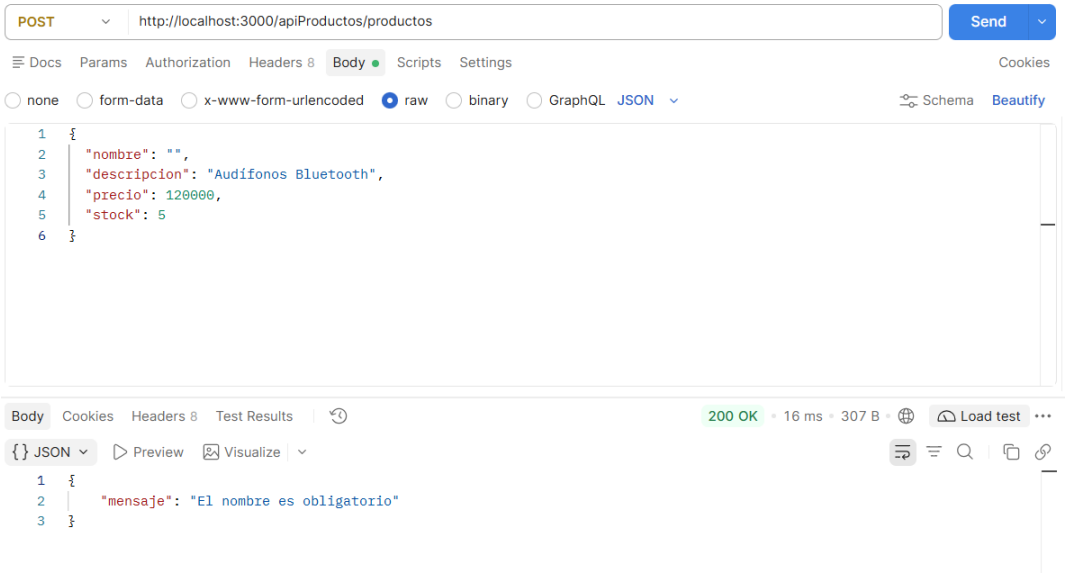
- **Registrar tres productos con diferentes precios y cantidades.**





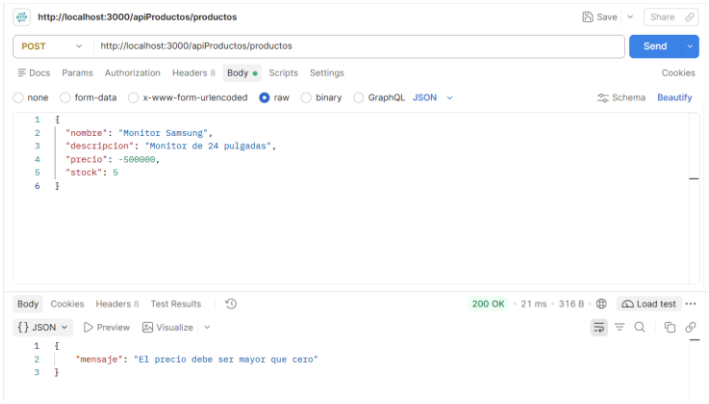
Se registraron tres productos correctamente, cada uno con diferentes precios y cantidades de inventario.

- **Intentar registrar un producto sin nombre.**

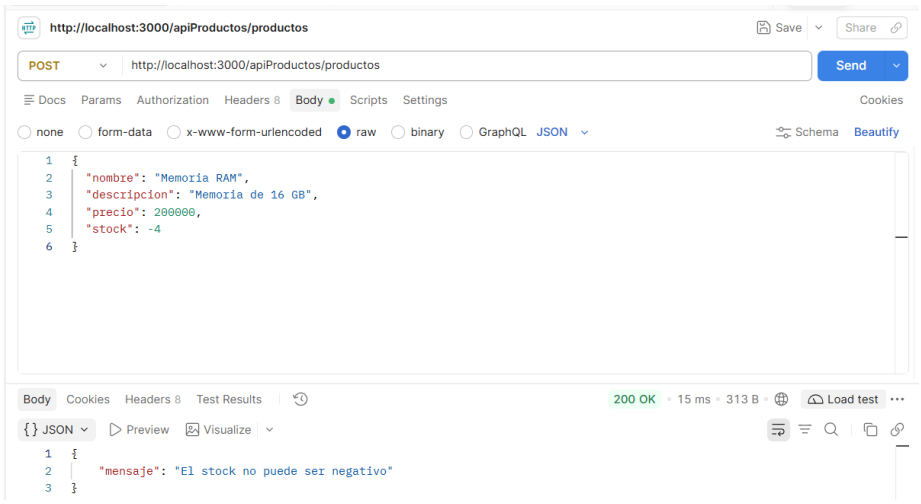


Si el servidor no tiene validaciones, puede permitir registrar un producto sin nombre. Por eso, se debería comprobar que este campo no esté vacío antes de guardar los datos.

- **Enviar un precio negativo y explicar cómo debería validarse.**



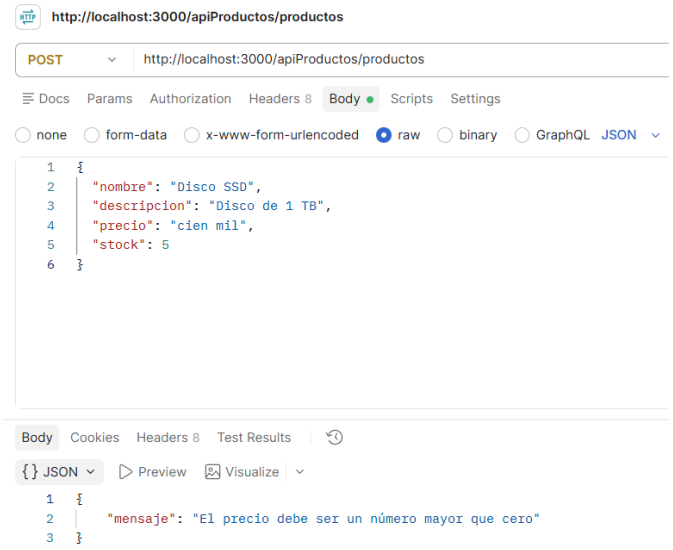
- **Enviar un stock negativo y explicar cómo debería validarse.**



El stock negativo no debería permitirse porque representa una cantidad de productos que no existe. El servidor debería validar que el stock sea igual o mayor que cero.

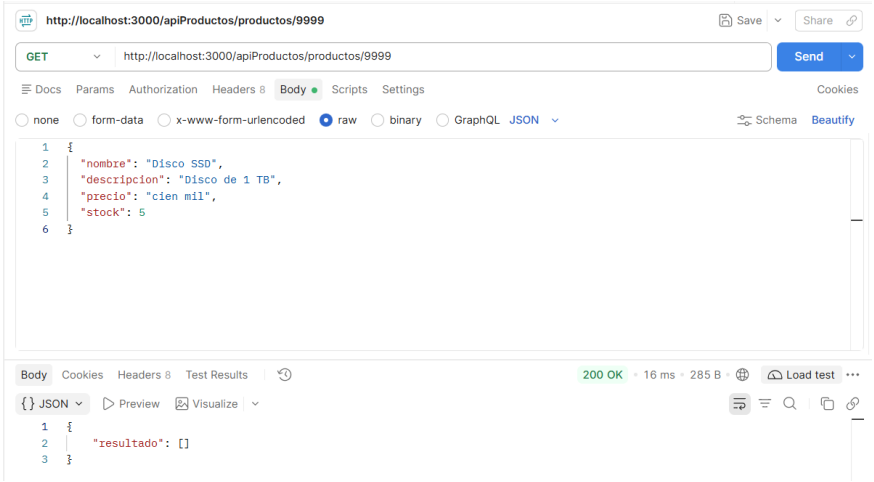
- **Enviar el precio como texto no numérico.**

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS



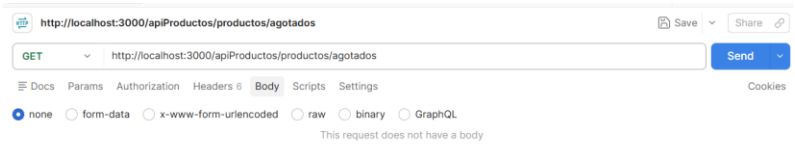
Al enviar el precio como texto no numérico ("cien mil"), el servidor rechaza el registro porque el precio debe ser un valor numérico. Se agregó una validación para evitar que se guarden datos incorrectos.

- **Buscar un producto inexistente.**



Al buscar un producto con un identificador que no existe, el servidor devuelve un resultado vacío porque no encuentra ese registro en la base de datos.

- **Consultar productos agotados y productos con bajo inventario.**





El sistema consulta y muestra los productos que tienen cero unidades disponibles en el inventario, es decir, los productos que están agotados.

Actividad 5. Integración de rutas y conexión

Enunciado: Organice los endpoints utilizando Express Router. Cada módulo debe contar con un archivo de rutas independiente. Los Controllers deben ser instanciados desde sus respectivos archivos Routes.

El archivo app.js debe:

1. Importar Express, CORS y MySQL2.

```
const express=require("express")
const cors=require("cors")
const app=express();
const PORT=3000;
const mysql=require("mysql2")
```

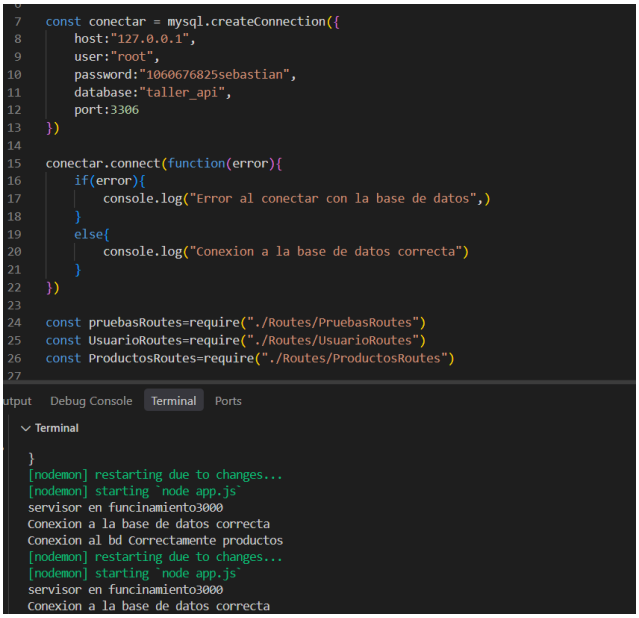
2. Crear la aplicación Express.

```
const express=require("express")
const cors=require("cors")
const app=express();
```

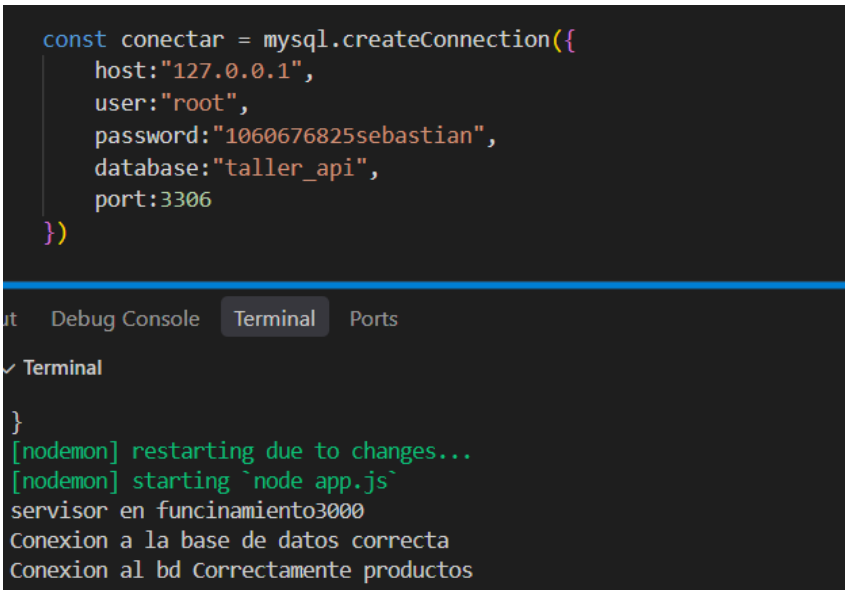
3. Configurar express.json() y cors().

```
app.use(express.json());
app.use(cors());
```

4. Crear la conexión a MySQL en el mismo app.js.



5. Compartir la conexión con los Controllers o hacerla disponible para las rutas de usuarios y productos.



6. Registrar las rutas de prueba, usuarios y productos.

```
const pruebasRoutes=require("./Routes/PruebasRoutes")
const UsuarioRoutes=require("./Routes/UsuarioRoutes")
const ProductosRoutes=require("./Routes/ProductosRoutes")


app.use("/apiPrueba",pruebasRoutes)
app.use("/apiUsuario",UsuarioRoutes)
app.use("/apiProductos",ProductosRoutes)
```

7. Iniciar el servidor en el puerto 3000.

```
const PORT=3000;
```

```
app.listen(PORT, function(){
    console.log('servisor en funcinamiento' + 3000)
})
```

Condición técnica: utilizar require() y module.exports. No crear una clase para el servidor. La conexión MySQL debe permanecer en app.js.

Actividad 6. Identificación y solución de errores

Enunciado: Durante el desarrollo, provoque de manera intencional los siguientes escenarios. Observe el mensaje de error o el comportamiento, identifique su causa, corrija el código y registre la evidencia.

Situación provocada	Error o comportamiento que debe investigar
No instalar Express	Cannot find module 'express'.
Importar una ruta incorrecta	Cannot find module.
No configurar express.json()	req.body puede llegar como undefined cuando se envía JSON sin el middleware adecuado.
Ejecutar guardar() en vez de pasar el método	El método se ejecuta inmediatamente, antes de recibir la petición.
Usar bind() incorrectamente	Puede perderse el contexto de la instancia o no conservarse this.
No conectar con MySQL	Se presenta un error de conexión.

Escribir mal el nombre de una tabla	Error SQL, por ejemplo ER_NO_SUCH_TABLE.
Enviar NULL a una columna NOT NULL	Puede producir ER_BAD_NULL_ERROR.
Consultar un ID inexistente	La consulta devuelve cero filas; debe manejarse como no encontrado.
Invocar un método sin req/res	Puede aparecer Cannot read properties of undefined (reading 'body').

Archivo donde ocurrió

Error presentado: Cannot find module 'express'.

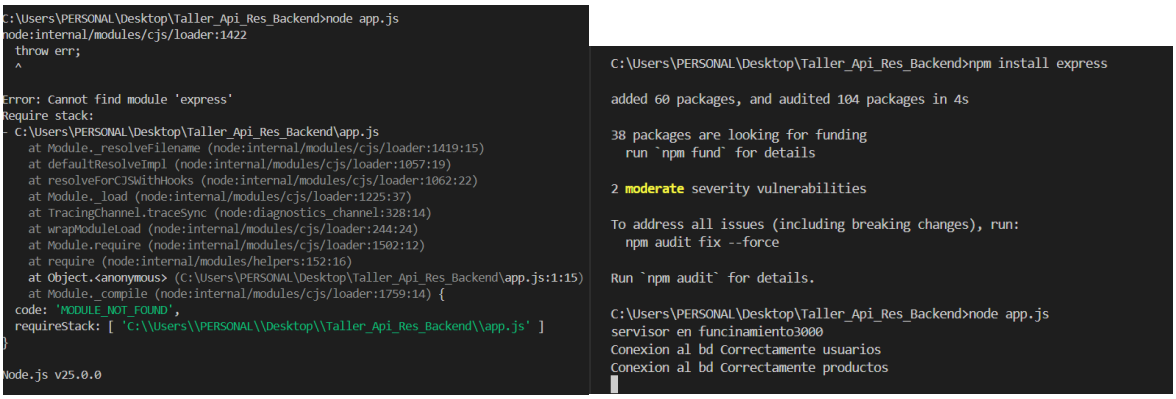
Archivo donde ocurrió: app.js

Línea aproximada: Línea donde se importa Express: const express = require("express").

Causa identificada: No estaba instalado el paquete Express en el proyecto.

Solución aplicada: Ejecuté el comando npm install express en la terminal para instalar nuevamente Express.

Evidencia (captura)



Error presentado: Cannot find module './Routes/PruebasRoute'.

Archivo donde ocurrió: app.js

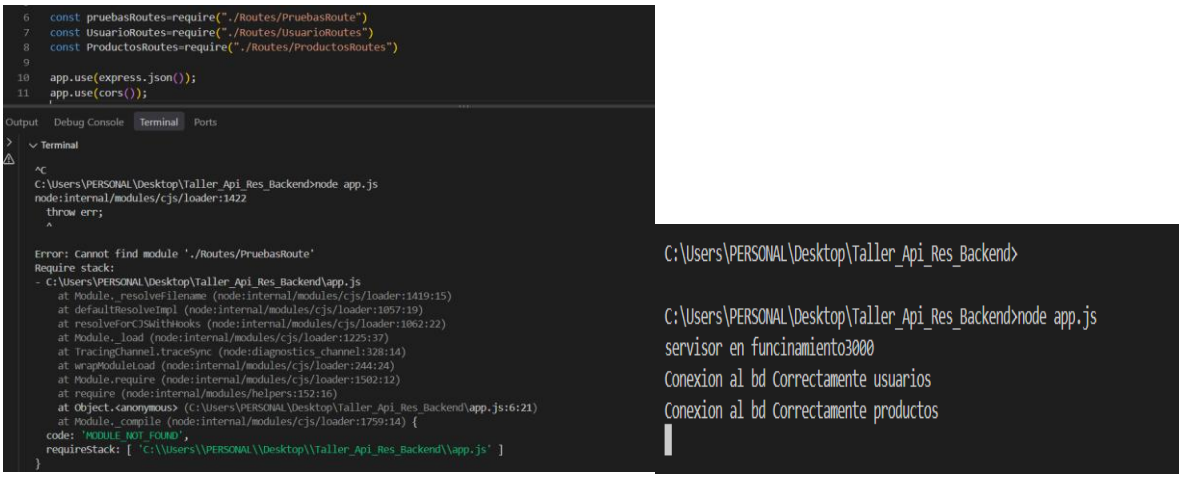
Línea aproximada: Línea donde se importa el archivo PruebasRoutes.

Causa identificada: Se escribió incorrectamente el nombre o la ruta del archivo que contiene las rutas de prueba.

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Solución aplicada: Corregí el nombre del archivo en require() y escribí correctamente ./Routes/PruebasRoutes.

Evidencia (captura)



Error presentado: req.body llegó como undefined al enviar los datos JSON.

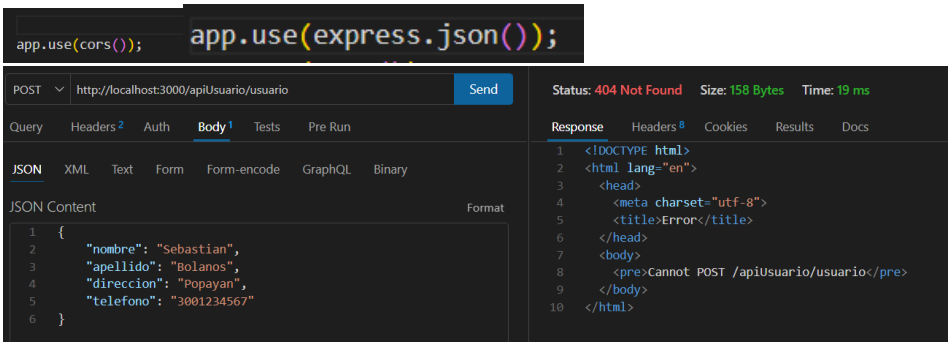
Archivo donde ocurrió: app.js y UsuariosController.js.

Línea aproximada: Línea de app.use(express.json()) en app.js.

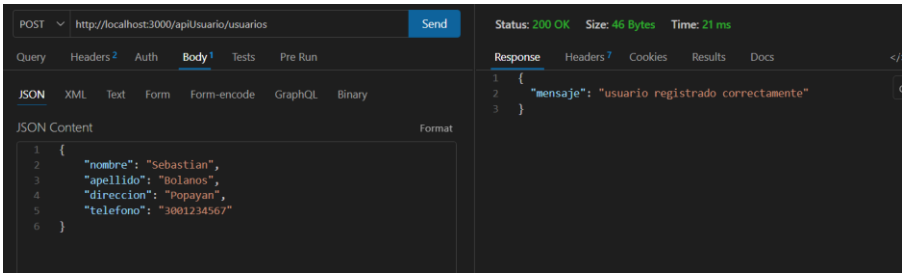
Causa identificada: No estaba configurado el middleware express.json(), necesario para recibir los datos enviados en formato JSON.

Solución aplicada: Agregué nuevamente app.use(express.json()) antes de registrar las rutas.

Evidencia (captura)



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS



Error presentado: El método guardar() se ejecutó inmediatamente al registrar la ruta.

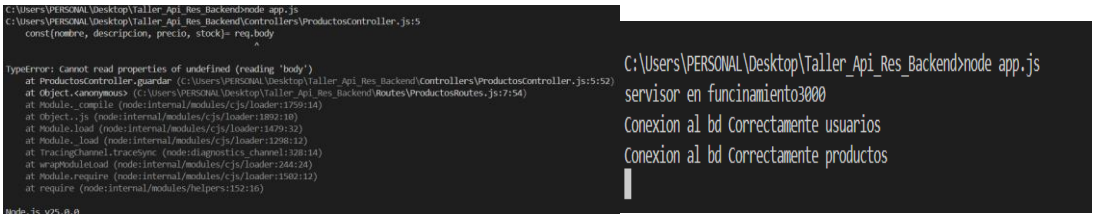
Archivo donde ocurrió: Routes/ProductosRoutes.js.

Línea aproximada: Línea donde se registra la ruta POST de productos.

Causa identificada: Se colocaron paréntesis después de guardar(), provocando que el método se ejecutara inmediatamente en lugar de esperar una petición.

Solución aplicada: Se eliminó la llamada directa y se utilizó bind(productosController) para pasar el método conservando el contexto de la instancia.

Evidencia (captura)



Error presentado: No se pudo establecer la conexión con la base de datos MySQL.

Archivo donde ocurrió: conexion/bdProductos.js

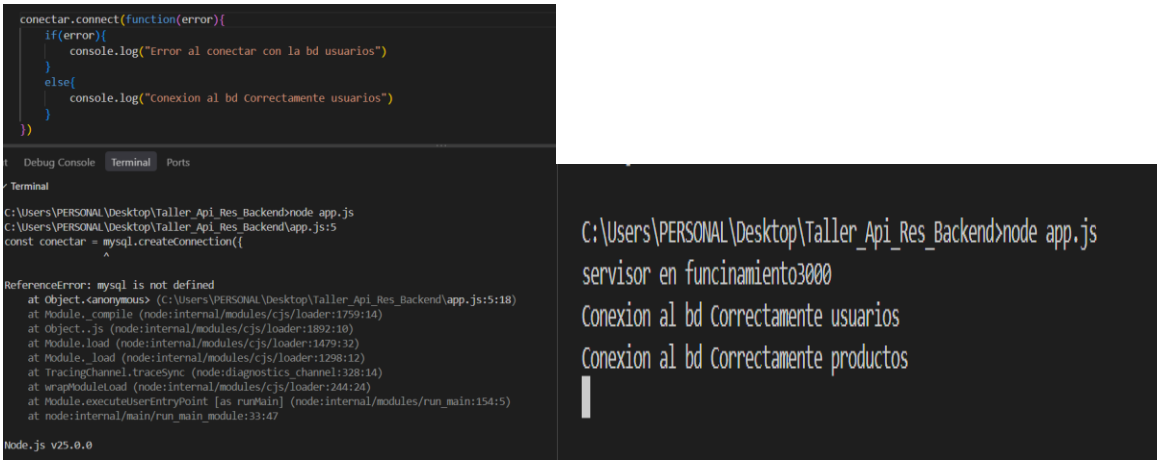
Línea aproximada: Línea 5, donde se configura el puerto de conexión.

Causa identificada: Se configuró un puerto incorrecto (3307) en lugar del puerto que utiliza MySQL (3306).

Solución aplicada: Se corrigió el puerto a 3306 para permitir la conexión con la base de datos.

Evidencia:

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS



Error presentado: Error al consultar los productos en la base de datos.

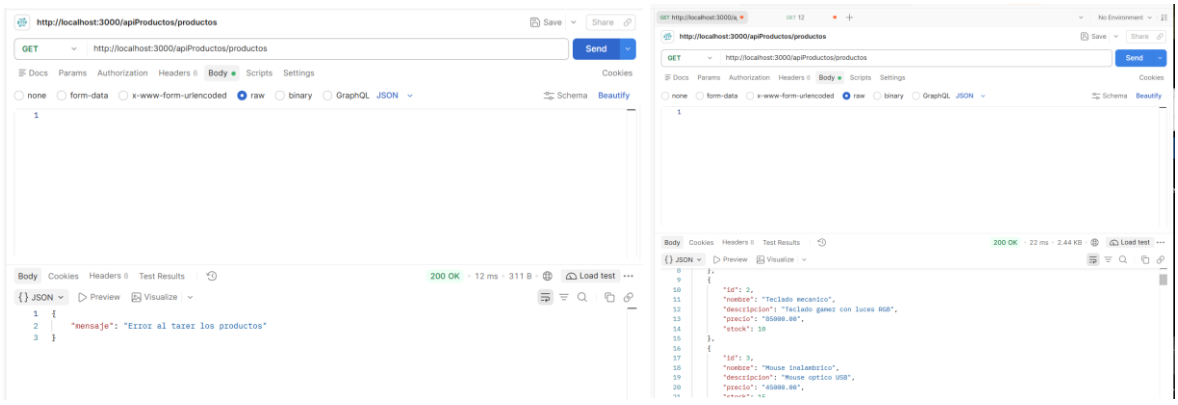
Archivo donde ocurrió: ProductosController.js

Línea aproximada: En la línea donde se encuentra la consulta SQL del método traer().

Causa identificada: Se escribió incorrectamente el nombre de la tabla, colocando tabla_productos en lugar de tabla_producto.

Solución aplicada: Se corrigió el nombre de la tabla en la consulta SQL.

6. Evidencia:



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

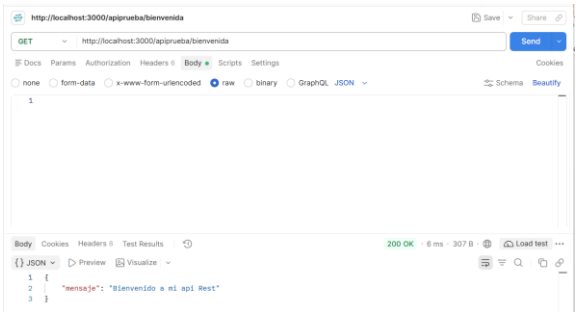
Actividad 7. Pruebas integrales con Postman

Enunciado: Ejecute y documente las pruebas de todos los endpoints. Utilice solicitudes correctas e incorrectas y conserve evidencia de las respuestas.

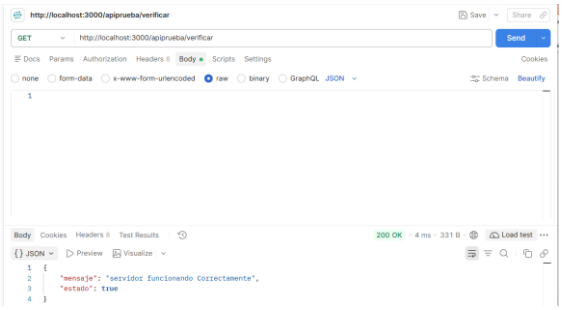
Módulo	Método	Funcionalidad
Prueba	GET	Bienvenida
Prueba	GET	Verificar servidor
Prueba	POST	Recibir datos
Prueba	POST	Sumar dos números
Usuarios	POST	Registrar
Usuarios	GET	Consultar todos
Usuarios	GET	Buscar por ID
Usuarios	PUT	Actualizar
Usuarios	DELETE	Eliminar
Productos	POST	Registrar
Productos	GET	Consultar todos
Productos	GET	Buscar por ID
Productos	PUT	Actualizar
Productos	DELETE	Eliminar
Productos	GET	Consultar agotados
Productos	GET	Consultar bajo inventario

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

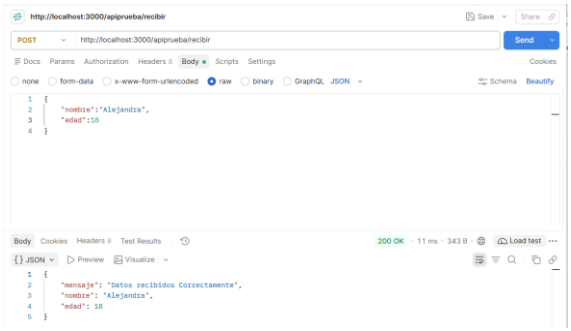
- **Prueba – GET – Bienvenida.**



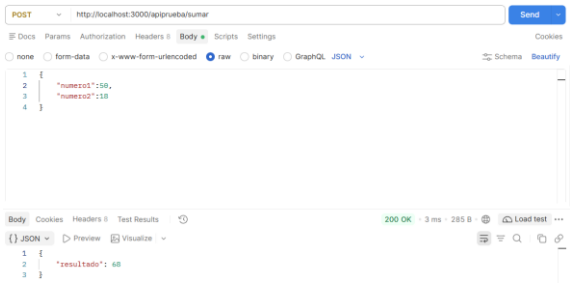
- **Prueba – GET – Verificar servidor.**



- **Prueba – POST – Recibir datos.**

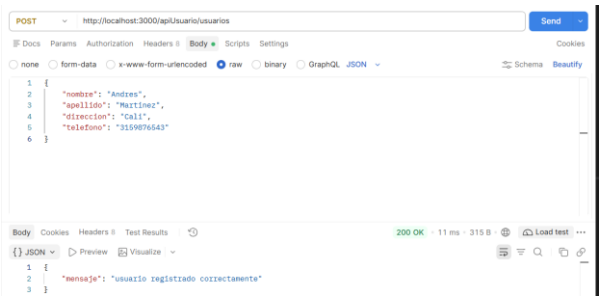


- **Prueba – POST – Sumar dos números.**

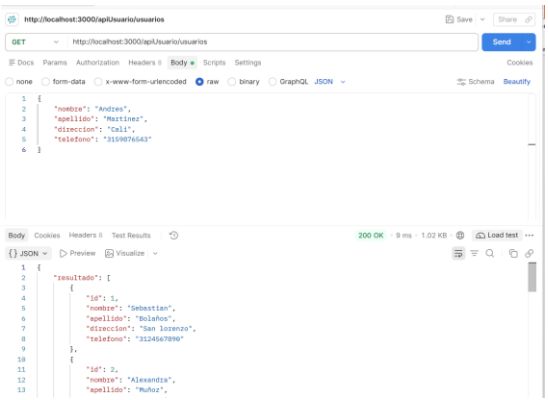


PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

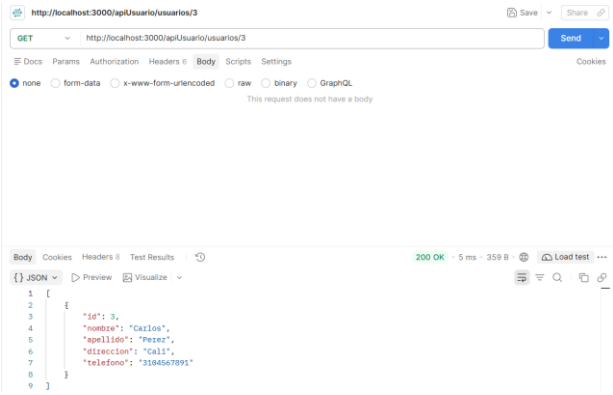
- **Usuarios – POST – Registrar.**



- **Usuarios – GET – Consultar todos.**

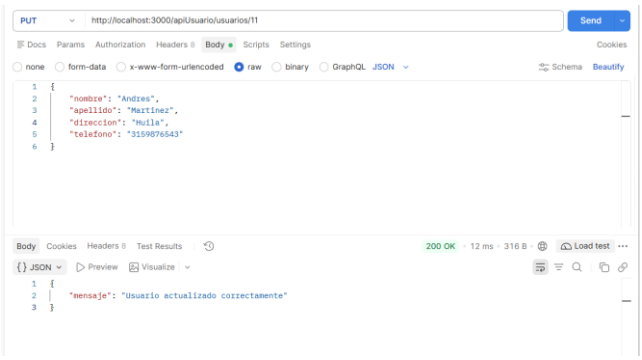


- **Usuarios – GET – Buscar por ID.**

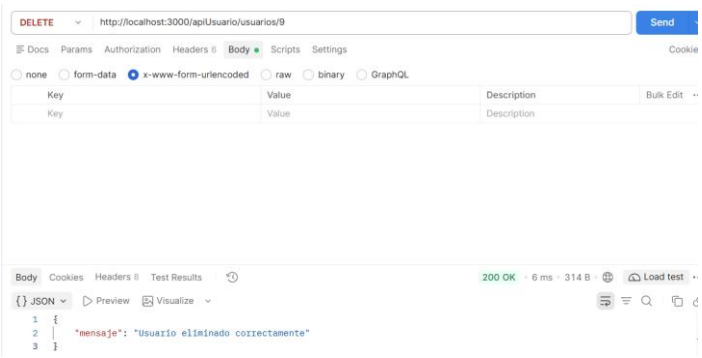


PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

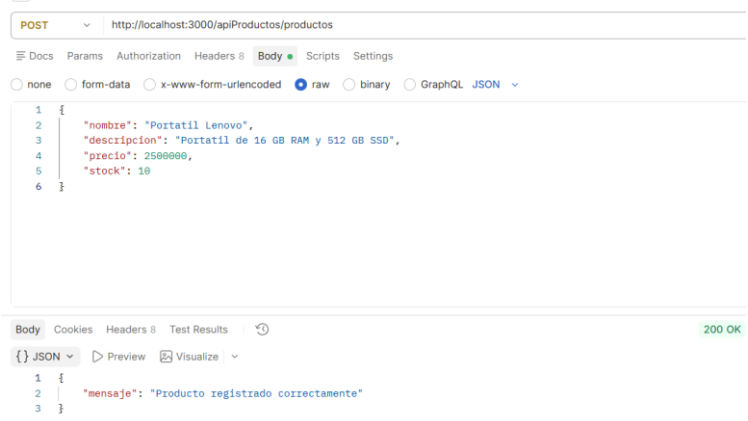
- **Usuarios – PUT – Actualizar.**



- **Usuarios – DELETE – Eliminar.**

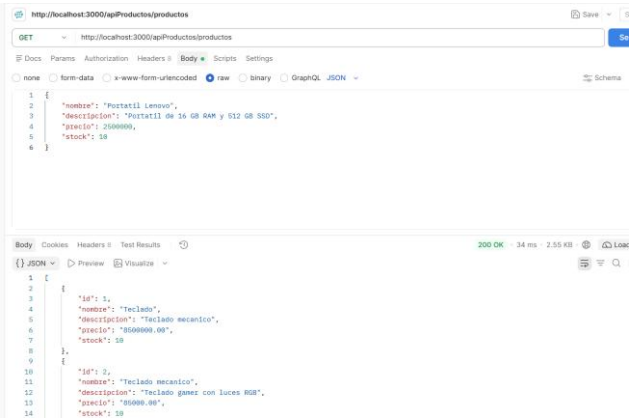


- **Productos – POST – Registrar.**

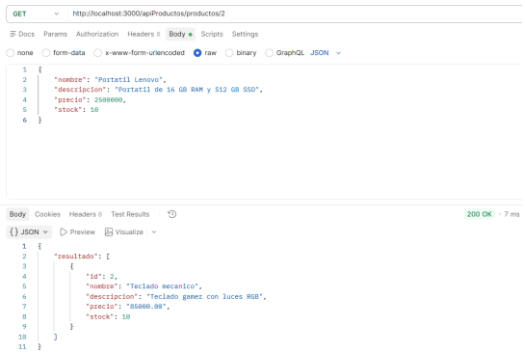


PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

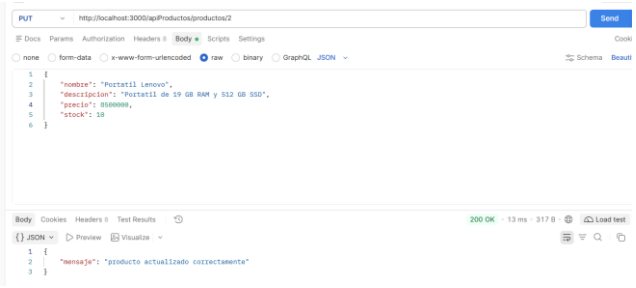
- **Productos – GET – Consultar todos.**



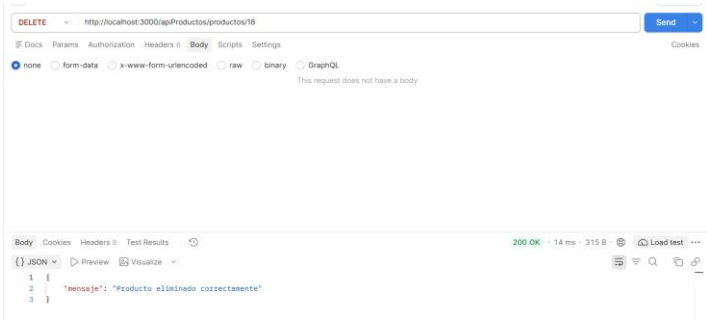
- **Productos – GET – Buscar por ID.**



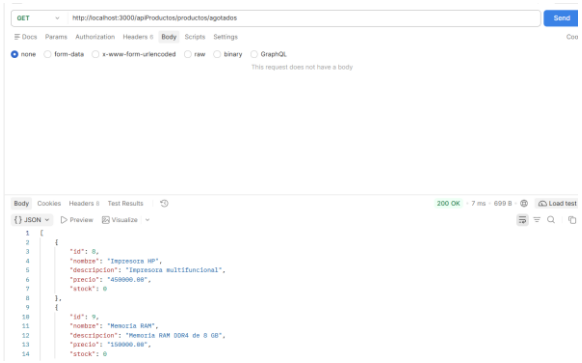
- **Productos – PUT – Actualizar.**



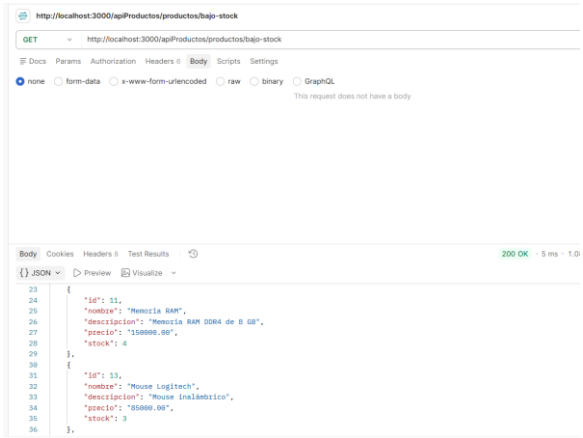
- **Productos – DELETE – Eliminar.**



- **Productos – GET – Consultar agotados.**



- **Productos – GET – Consultar bajo inventario.**



8. Preguntas de análisis y sustentación

1. ¿Qué es Node.js y cuál es su función en el proyecto?

Node.js es un entorno que permite ejecutar JavaScript fuera del navegador. En el proyecto lo utilizamos para crear y ejecutar el servidor backend.

2. ¿Qué es Express y qué ventajas ofrece?

Express es un framework de Node.js que facilita la creación de servidores y API. Permite organizar las rutas y manejar las solicitudes de una manera más sencilla.

3. ¿Qué es una API REST?

Es una forma de comunicación entre el frontend y el backend mediante solicitudes HTTP para consultar, registrar, actualizar y eliminar información.

4. ¿Qué diferencia existe entre GET, POST, PUT y DELETE?

- GET: permite consultar información.
- POST: permite registrar información nueva.
- PUT: permite actualizar información existente.
- DELETE: permite eliminar información.

5. ¿Qué función cumple express.json()?

Permite que Express pueda recibir y leer los datos que llegan en formato JSON en el cuerpo de una solicitud.

6. ¿Qué es una clase en JavaScript?

Es una estructura que permite organizar el código y crear objetos que comparten propiedades y métodos.

7. ¿Qué es un objeto y cómo se instancia?

Un objeto es una estructura que contiene información y funciones. Se instancia utilizando la palabra new seguida del nombre de la clase.

8. ¿Qué función cumple el constructor?

El constructor es un método especial que se ejecuta cuando se crea un objeto y permite inicializar sus propiedades.

9. ¿Qué diferencia existe entre un método tradicional y una propiedad con función flecha?

Un método tradicional tiene su propio comportamiento con this, mientras que una función flecha utiliza el this del contexto donde fue creada.

10. ¿Para qué se utiliza module.exports?

Se utiliza para exportar una clase, función u objeto y permitir que pueda utilizarse en otro archivo.

11. ¿Qué función cumple require()?

Permite importar módulos o archivos para utilizarlos en otro archivo de JavaScript.

12. ¿Qué responsabilidad tiene un Controller?

Se encarga de recibir las solicitudes, ejecutar la lógica necesaria y devolver una respuesta al usuario.

13. ¿Qué responsabilidad tiene Routes?

Se encarga de definir las rutas de la API y relacionarlas con los métodos correspondientes del Controller.

14. ¿Por qué la conexión MySQL se comparte desde app.js?

Porque permite centralizar la conexión a la base de datos y reutilizarla en los diferentes controladores, evitando crear conexiones innecesarias.

15. ¿Qué función cumplen los signos ? en las consultas SQL parametrizadas?

Son espacios reservados que permiten enviar los valores de forma separada a la consulta SQL, ayudando a evitar la inyección SQL.

16. ¿Por qué es importante controlar los errores en una API?

Porque permite identificar y solucionar problemas, evitar fallos inesperados y entregar respuestas claras cuando ocurre un error.